



Essay / Assignment Title:

Cloud-Enabled AI Vision Framework for Drones and Mobile Robots in Crop
Disease Classification and Autonomous Robotic Simulation

Programme title:

MSc Artificial Intelligence

Prepared by:

Amaan Mohammed Khalander

Module:

Advanced Robotics Automation and Computer Vision

Year:

Term 2 – 2026

Submission Date:

23 / September / 2026

Issued by:

Dr. Fahmid Al Farid

Learner declaration

Item	Response
Word count	WORD COUNT HERE IN NUMBER (1424)
Use of proof-reader/proof-reading service	NO
If yes, specify which service	N/A
I confirm that I submit this work as my own work and have cited all sources used	YES
I confirm that I have followed guidance on the acceptable use of AI tools for this assignment	YES
Where use of appropriately cited AI tools is permitted, I confirm that I have cited in accordance with the UCA Harvard Referencing Standard	YES

Student signature	Date
	10 / 06 / 2026

Source Code and Relevant Project Links:

Source Code

<https://github.com/Amaan9136/bsbi-assignments/blob/main/Sem1/Machine%20Learning%20Fundamentals/project-assignment/main.ipynb>

Report

<https://github.com/Amaan9136/bsbi-assignments/blob/main/Sem2/Advanced%20Robotics%20Automation%20and%20Computer%20Vision/Report.pdf>

Table of Contents

Executive Summary	6
Introduction.....	7
Task 1 - Autonomous Drone-Based Vision System for Crop Health Monitoring and Disease Classification.....	8
3.1 Project Definition.....	8
3.2 Literature Review.....	8
3.3 Design and Implementation	9
3.4 Execution and Testing.....	11
3.5 Evaluation and Reflection.....	13
Task 2 - Vision-Guided Autonomous Navigation in ROS	16
4.1 Project Definition.....	16
4.2 Design and Implementation	18
4.3 Execution and Testing.....	19
4.4 Evaluation and Reflection.....	21
Combined Evaluation and Reflection	23
Concluding Remarks.....	23
Acknowledgement	24
References.....	25

Table of Figures

Figure 1: Distribution of images across the 38 PlantVillage classes, illustrating the class imbalance referred to above.....	9
Figure 2: Grid of augmented training images with labels, illustrating the transformations described above.....	10
Figure 3: MobileNetV2 architecture overview, showing the pretrained backbone with the original 1000-class head replaced by a 38-class linear layer	11
Figure 4: Training and validation loss and accuracy across all eight epochs.	12

Figure 5: Sample predictions on validation images, showing predicted label, true label and model confidence.	13
Figure 6: Full 38-class confusion matrix on the validation set.	14
Figure 7: TheConstruct.ai ROSject workspace (rosject 1049584), showing the turtlebot3_vision_nav package files and a terminal session used to inspect /cmd_vel, /target_offset and /target_visible directly with ros2 topic echo.	16
Figure 8: The waffle_pi TurtleBot3 spawned in turtlebot3_world, showing the entity tree and simulation configuration.	17
Figure 9: The simulated scene during placement of a target marker and lighting within the world.	18
Figure 10: The perception-control pipeline, showing the two nodes, the topics connecting them, and the debug image branch.	19
Figure 11: Synthetic test frames with the computed offset, visibility flag and resulting velocity command for each scenario.	20

List of Tables

Table 1: Final training epochs.....	12
Table 2: Final validation performance (Task 1)	13
Table 3: Offline verification summary	20
Table 4: Live ROSject verification (ros2 topic echo, single query each, shown in Figure 7) .	21

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname:

AMAAN MOHAMMED KHALANDER

A handwritten signature in black ink, appearing to read 'Amaan MK', with a stylized, cursive script.

Date: 27 / May / 2026

Executive Summary

This report covers two related projects carried out for the module Advanced Robotics Automation and Computer Vision. The first is a deep learning vision system for crop health monitoring, trained on the PlantVillage dataset and adapted through augmentation to approximate imagery from a low altitude agricultural drone, accounting for viewpoint change, motion blur and variable outdoor lighting. A MobileNetV2 backbone, pretrained on ImageNet, was fine-tuned on 54,305 leaf images across 38 classes using a stratified 80/20 split. Over eight training epochs, the model achieved a final validation accuracy of 99.11%, with macro precision of 98.64%, macro recall of 99.06% and a macro F1 score of 98.83%. The resulting classifier has around 2.27 million trainable parameters, in line with the goal of onboard inference on embedded drone hardware.

The second project implements vision guided navigation for a simulated TurtleBot3 robot within a ROS 2 based Gazebo environment, using OpenCV colour thresholding to detect a marker and a rule-based controller to search, turn and approach it. The perception and control logic was verified offline against seven synthetic test frames, confirming correct behaviour across centred, off-centre, near, far, absent and distractor scenarios. A live query of the deployed ROSject on TheConstruct.ai reproduced the same proportional turning behaviour to five decimal places. Together, the two projects show how a learned, offline classification approach and a classical, real time control approach each suit the computational and latency demands of their respective robotic context.

Introduction

Robotic vision systems now underpin two areas of considerable practical importance. In precision agriculture, automated and early detection of crop disease can materially affect yield and food security, while in autonomous mobile robotics, a robot's capacity to interpret its surroundings visually determines how far it can operate without continuous human supervision. This report addresses both areas through two tasks issued under a single assignment brief for the module Advanced Robotics Automation and Computer Vision, and presents them together in one document.

Task 1 develops a drone-oriented plant disease classification system, converting raw aerial imagery into an actionable assessment of crop health with sufficient computational efficiency to run on the drone itself. Task 2 develops a vision guided navigation system for a simulated ground robot, in which a mobile platform interprets its camera feed to locate a target and adjusts its own movement accordingly.

Although the two tasks operate in different physical contexts, both convert pixel level image data into a decision a robotic system can act upon, under practical constraints of computational cost. Task 1 is further aligned with United Nations Sustainable Development Goal 2, Zero Hunger, which identifies scalable crop disease detection as a contributor to food security. The two tasks are reported as one document because they share a common module, a common cloud-based development approach and a common design principle, discussed directly in the combined evaluation section.

Task 1 - Autonomous Drone-Based Vision System for Crop Health Monitoring and Disease Classification

3.1 Project Definition

The problem addressed by Task 1 is the automated classification of plant leaf images into healthy or diseased categories, adapted to reflect the imaging conditions of a camera mounted on an agricultural survey drone rather than a handheld camera used at close range. The objective is to build and evaluate a convolutional neural network capable of this classification while remaining lightweight enough to be considered for deployment on embedded hardware carried by the drone, such as a Jetson Nano, rather than requiring imagery to be transmitted to a remote server for processing.

The scenario assumed is a precision agriculture survey in which a multirotor drone flies a defined route over a field at low altitude, capturing RGB imagery at intervals, so the vision module can flag regions showing signs of disease and let a grower prioritise manual inspection rather than checking the entire field by hand.

3.2 Literature Review

A number of recent papers inform the choices made in this project, spanning three related areas: deep learning for plant disease classification, AI applied to agricultural drones, and lightweight CNNs built for edge devices.

- **Deep learning in plant disease classification.** Mohanty, Hughes and Salathe (2016) were among the first to show that deep CNNs trained on PlantVillage could exceed 99% accuracy, though their work used the dataset's original close range, controlled photographs. Pal et al. (2025) built on this with a lightweight, explainable CNN that reached 99.47% on PlantVillage and 97.73% on a much larger 58 class dataset, which suggests that smaller models do not necessarily sacrifice accuracy for efficiency.
- **AI in agricultural drones.** Manoj et al. (2025) applied disease detection directly to imagery captured by drones over large fields, and a wider review of AI enabled UAV systems (arXiv, 2023) lays out the sensing and onboard processing limits that come with aerial platforms specifically. Both papers point to something this project could not fully replicate: real UAV imagery behaves differently from ground level photography in terms of scale, viewpoint and motion blur, and that gap is only approximated here through augmentation rather than closed with genuine aerial data.

- **Lightweight CNNs for edge devices.** Sandler et al. (2018) introduced the inverted residual structure that MobileNetV2 is built on, which is the backbone used in this project. Kumar et al. (2025) benchmarked lightweight CNNs across 101 disease classes and 33 crops and found the accuracy versus efficiency trade off still holds even at that scale. Jlassi et al. (2024) applied MobileNetV2 transfer learning with a reweighting scheme to potato leaf disease classification specifically to handle imbalanced data, and their results held up well across underrepresented classes, which is directly relevant given that PlantVillage itself is not perfectly balanced.

Taken together, these papers show that lightweight CNNs are usually tested on curated, close-range datasets rather than on real aerial captures. The augmentation pipeline used in this project, which tries to mimic drone specific conditions like viewpoint change, motion blur and variable lighting, is a direct attempt to close part of that gap, though it stops short of testing against genuine UAV imagery, a limitation picked up again in Section 3.5.

3.3 Design and Implementation

The PlantVillage dataset was used, accessed via the Hugging Face datasets library at mohanty/PlantVillage using the colour configuration, mirroring the original repository (mohanty/PlantVillage, 2016; PlantVillage-Dataset, 2016), and comprising 54,305 labelled leaf images across 38 crop and disease classes, a widely used baseline in plant pathology research (Mohanty, Hughes and Salathe, 2016).

Class counts were inspected prior to splitting, since PlantVillage is not perfectly balanced; some disease categories are represented by substantially fewer images than the most common classes.

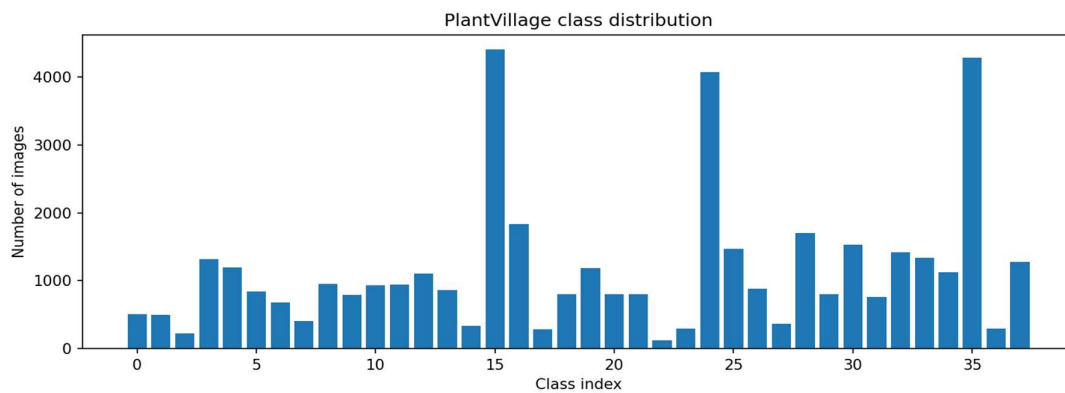


Figure 1: Distribution of images across the 38 PlantVillage classes, illustrating the class imbalance referred to above.

A stratified split was used to divide the data into 43,444 training images and 10,861 validation images, preserving the class proportions of the full dataset in both subsets.

Preprocessing resized all images to 224 by 224 pixels and normalised pixel values using the mean and standard deviation associated with ImageNet, consistent with the pretrained MobileNetV2 weights used as a starting point. Augmentation applied only to the training subset combined random rotation, random affine translation and scaling, colour jitter in brightness, contrast and saturation, mild Gaussian blur, random perspective distortion and horizontal flipping.



Figure 2: Grid of augmented training images with labels, illustrating the transformations described above.

These transformations were chosen to approximate conditions associated with aerial capture: rotation and perspective distortion approximate viewpoint variation from a moving drone, blur approximates motion and vibration during flight, and colour jitter approximates the wider range of outdoor lighting a drone encounters relative to the more controlled conditions of the original photography.

MobileNetV2 was selected as the backbone, pretrained on ImageNet and fine-tuned on PlantVillage, with the thousand class output layer replaced by a linear layer for the 38 PlantVillage classes, giving a final model of approximately 2.27 million trainable parameters.

MobileNetV2’s defining feature is the inverted residual block with linear bottlenecks, which uses depthwise separable convolutions to substantially reduce parameter count relative to standard architectures of comparable depth (Sandler et al., 2018), directly suiting a drone with tight onboard compute, memory and power constraints. The overall structure is summarised in Figure 3, showing the pretrained backbone feeding into the replaced classification head. Pal et al. (2025) report MobileNet family architectures retaining accuracy above 99 percent on PlantVillage while remaining edge deployable, and Kumar et al. (2025) similarly benchmark lightweight CNNs across a larger class set, both supporting the choice made here.

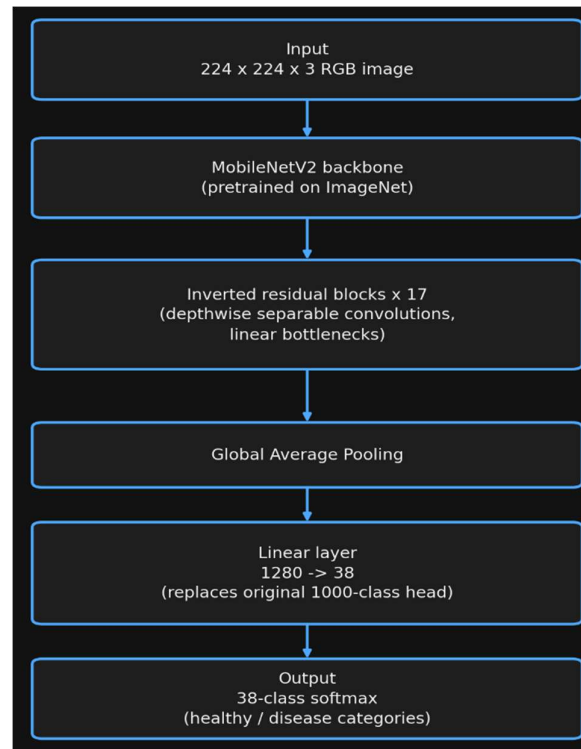


Figure 3: MobileNetV2 architecture overview, showing the pretrained backbone with the original 1000-class head replaced by a 38-class linear layer.

The implementation was split into distinct modules rather than a single script: configuration, data loading and augmentation, model construction, the fine-tuning loop, evaluation, inference and plotting, with training using cross entropy loss, the Adam optimiser and a learning rate scheduler that reduces the learning rate when validation loss plateaus. This separation let the same underlying logic run identically from the notebook or from a local script, and let components such as the augmentation pipeline be checked independently before the full run.

3.4 Execution and Testing

Training was run locally by executing Task1_Drone_Crop_Disease_Classification.ipynb on a machine with a single CUDA GPU visible to PyTorch, for eight epochs with a batch size of 32 and a learning rate of $3e-4$. The notebook is publicly available at https://github.com/Amaan9136/bsbi-assignments/blob/main/Sem2/Advanced%20Robotics%20Automation%20and%20Computer%20Vision/task1-code/notebook/Task1_Drone_Crop_Disease_Classification.ipynb.

A thermal safeguard built into the training loop paused training whenever the GPU reached 70°C and resumed once it cooled to 45°C, a precaution specifically aimed at a local GPU

running for an extended period, which triggered repeatedly given the length of the run. Total training time was 73.6 minutes, of which 48.5 minutes were spent paused for cooldown, leaving roughly 25 minutes of active computation.

Table 1 summarises the final four training epochs, by which point the model, benefiting from ImageNet pretraining, was already converging.

Table 1: Final training epochs

Epoch	Train loss	Train accuracy	Val loss	Val accuracy
5	0.0612	0.9808	0.0598	0.9810
6	0.0522	0.9831	0.0339	0.9888
7	0.0520	0.9831	0.0533	0.9818
8	0.0485	0.9843	0.0281	0.9911

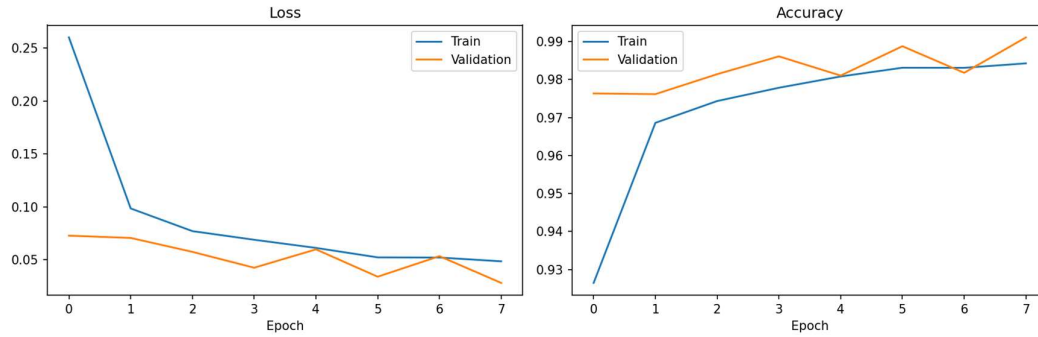


Figure 4: Training and validation loss and accuracy across all eight epochs.

Following training, the model was evaluated on the held-out validation set, and a grid of randomly sampled predictions with true label, predicted label and confidence was generated to inspect qualitative behaviour.



Figure 5: Sample predictions on validation images, showing predicted label, true label and model confidence.

3.5 Evaluation and Reflection

Table 2 summarises the final evaluation metrics on the held-out validation set.

Table 2: Final validation performance (Task 1)

Metric	Value
Accuracy	99.11%
Macro precision	98.64%
Macro recall	99.06%
Macro F1 score	98.83%
Best validation accuracy during training	99.11% (reached at epoch 8)
Total trainable parameters	2,272,550

The relationship between training and validation loss across the run, visible in Figure 4, does not indicate overfitting; validation loss tracked training loss closely and reached its lowest point at the final epoch alongside the best validation accuracy, which is consistent with the regularising effect of the augmentation pipeline.

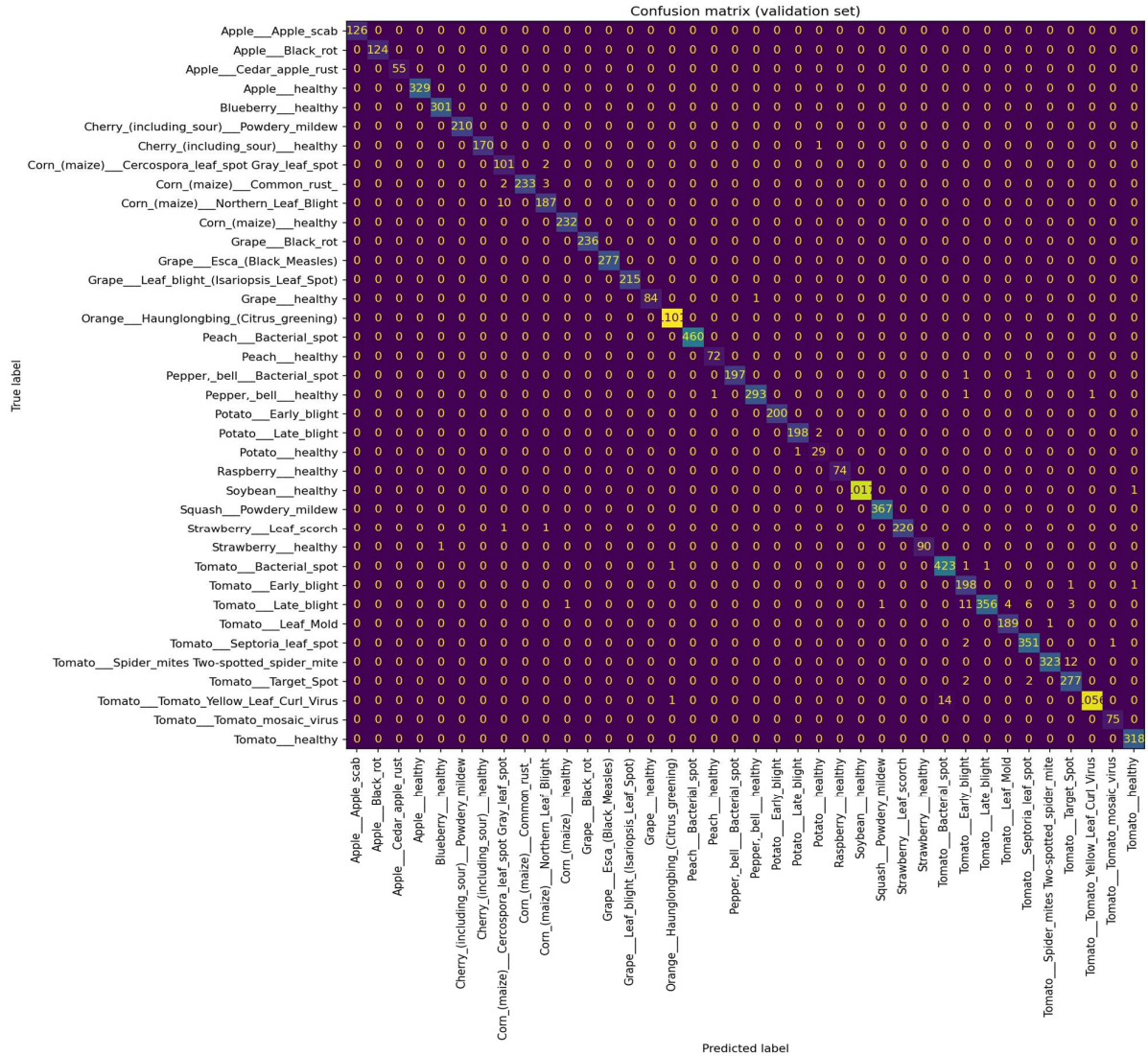


Figure 6: Full 38-class confusion matrix on the validation set.

The confusion matrix indicates that the small number of remaining misclassifications are concentrated among visually similar classes rather than spread randomly across the label set. The most frequent confusions were Tomato Yellow Leaf Curl Virus predicted as Tomato Bacterial spot (14 instances), Tomato Two-spotted spider mite predicted as Tomato Target Spot (12), Tomato Late blight predicted as Tomato Early blight (11), Corn Northern Leaf Blight predicted as Corn Cercospora leaf spot Gray leaf spot (10), and Tomato Late blight predicted as Tomato Septoria leaf spot (6). Every confused pair belongs to the same crop species and shares a broadly similar lesion pattern, a plausible source of difficulty for a classifier relying on colour and texture cues alone.

A per-class ROC or precision-recall curve was not generated for this task. With 38 classes, a full set of ROC curves adds visual clutter without much added insight beyond what the confusion matrix and macro-averaged metrics already show, and the macro precision, recall and F1 scores reported in Table 2 summarise per-class performance more concisely across all 38 categories. The confusion matrix in Figure 6 was used instead as the primary tool to inspect where errors occur, since it shows exactly which class pairs are confused rather than only a threshold-independent trade-off curve.

Independently of these results, two design level limitations are identified. First, the aerial imaging conditions targeted by this task are simulated through augmentation of ground level, close range photographs rather than genuine drone captured imagery. This is a reasonable and commonly used proxy where aerial training data is unavailable, but it does not reproduce every characteristic of true UAV imagery, such as the systematic relationship between altitude and object scale; publicly available UAV specific datasets referenced in reviews of AI enabled UAV systems (arXiv, 2023) and in recent drone-based crop disease work (Manoj et al., 2025) offer a more direct path to closing this gap.

Second, class imbalance in PlantVillage means that, without intervention such as class weighted loss, the classifier risks weaker performance on rarer disease categories, which is also where correct detection matters most. The parameter count achieved is consistent with the goal of onboard, low power inference. A realistic path to field deployment would additionally require converting the model to a runtime format such as TensorFlow Lite or ONNX, integrating GPS metadata to map detections back onto the field, and validating against genuine UAV imagery before operational use.

Task 2 - Vision-Guided Autonomous Navigation in ROS

4.1 Project Definition

Task 2 addresses vision guided navigation for a simulated TurtleBot3 mobile robot, developed and run within TheConstruct.ai's ROS 2 based Gazebo simulation environment (TheConstruct.ai, 2024). The objective is to enable the robot to detect a coloured marker placed within its environment using its onboard RGB camera and to navigate toward that marker using simple, interpretable vision-based rules rather than a learned control policy.

Development and testing were carried out directly on TheConstruct.ai's browser based ROSject platform, inside ROSject vVGb1L3fxD (URL identifier 1049584), workspace ros2_ws. The free tier of TheConstruct.ai does not issue a permanent shareable URL for a ROSject; the project was instead made public, such that it can be located and viewed via its title, vVGb1L3fxD, in the platform's public index, which is the mechanism used here to satisfy the brief's public accessibility requirement. Figure 7 shows this workspace, including the turtlebot3_vision_nav package structure and a terminal session used to query the running nodes directly.

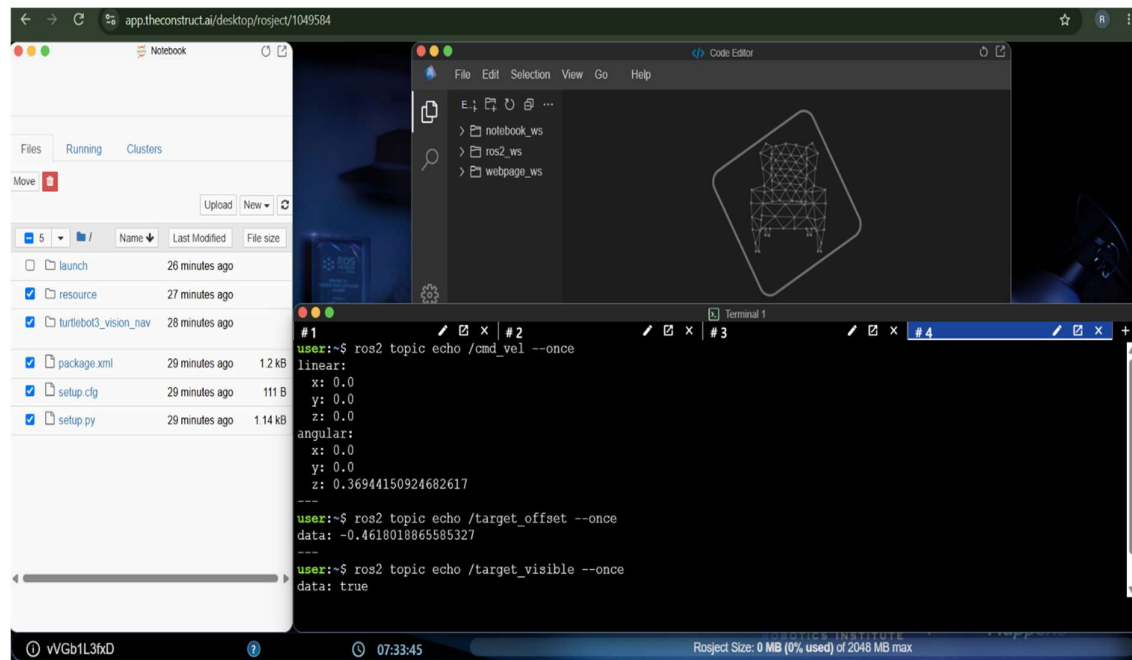


Figure 7: TheConstruct.ai ROSject workspace (rosject 1049584), showing the turtlebot3_vision_nav package files and a terminal session used to inspect /cmd_vel, /target_offset and /target_visible directly with ros2 topic echo.

The simulation environment is the standard turtlebot3_world provided with the TurtleBot3 packages, run in Gazebo Sim with the waffle_pi robot model (ROBOTIS, 2024), with a coloured marker placed at varying positions across test scenarios.

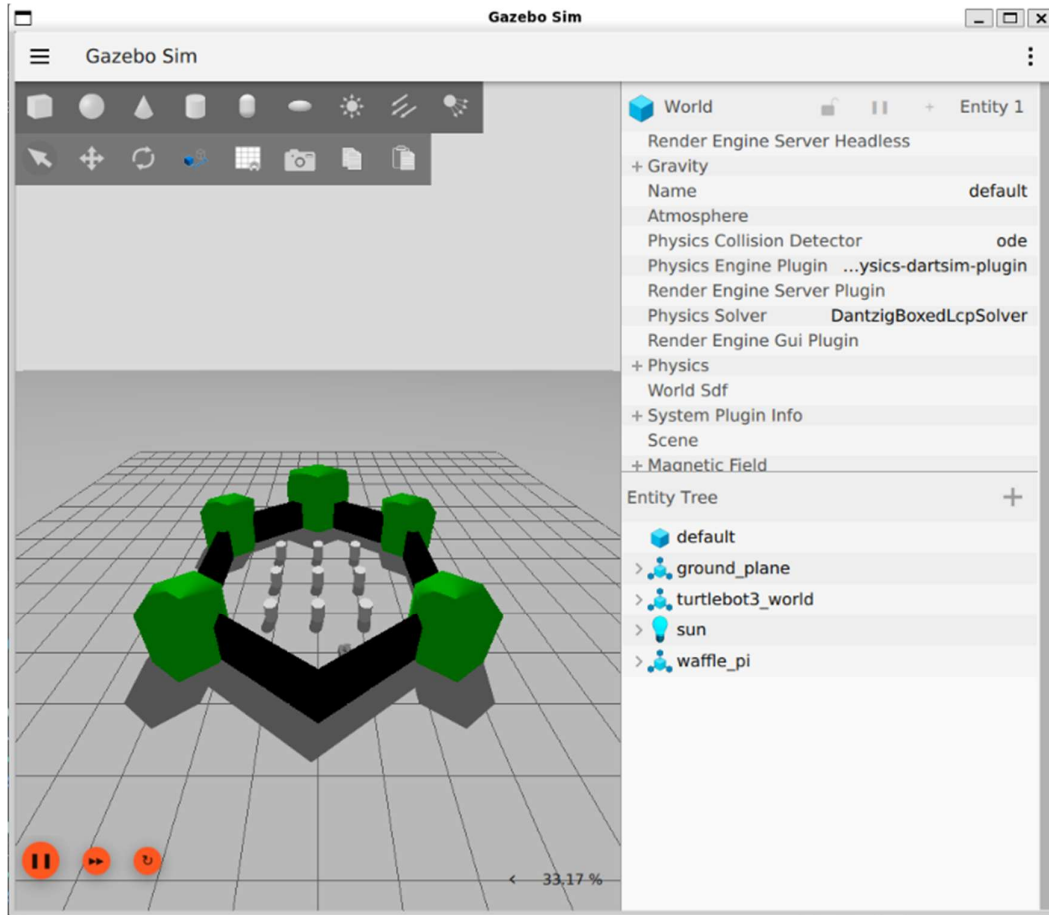


Figure 8: The waffle_pi TurtleBot3 spawned in turtlebot3_world, showing the entity tree and simulation configuration.

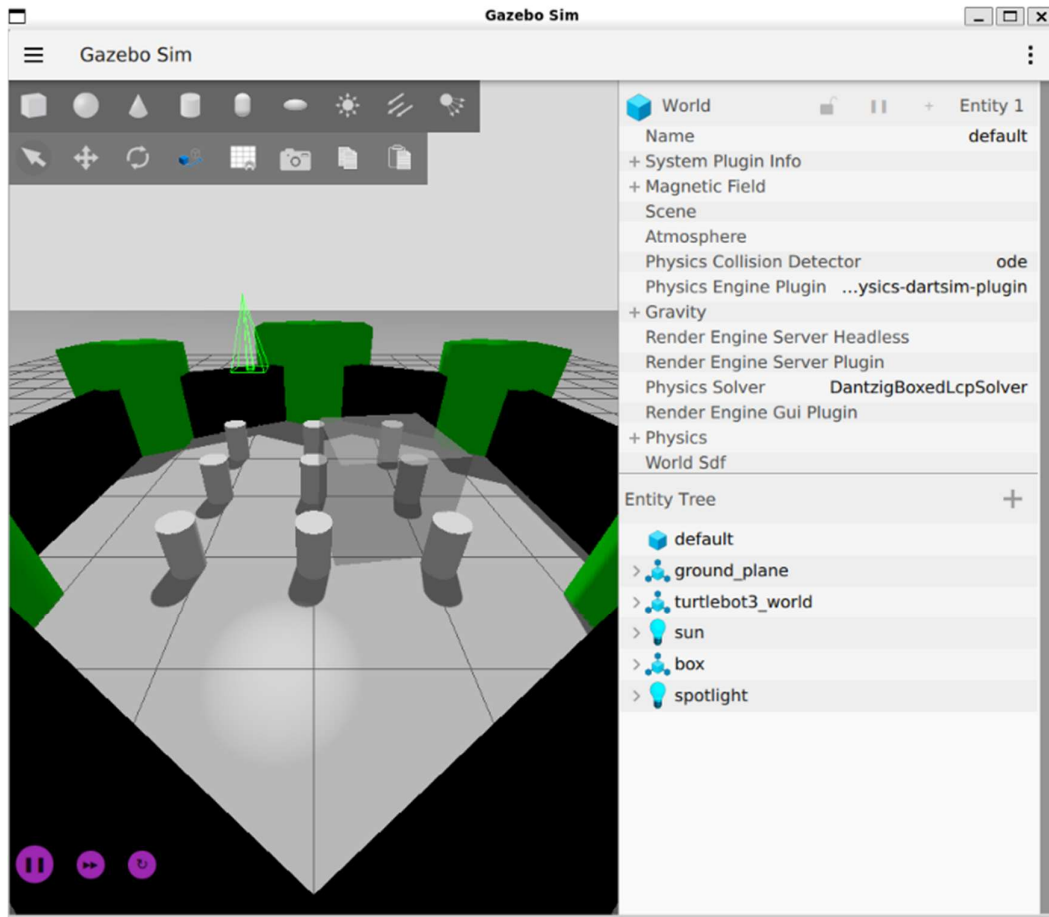


Figure 9: The simulated scene during placement of a target marker and lighting within the world.

4.2 Design and Implementation

The system is structured as two ROS 2 nodes connected by two lightweight topics, following the standard principle of separating perception from control so that each stage can be tested and reasoned about independently (Quigley et al., 2009). Both nodes are thin wrappers around plain Python modules, `vision_core.py` and `control_core.py`, containing no ROS dependency, so that the detection and control algorithms could be developed and verified outside of a running simulation before being tested against it.

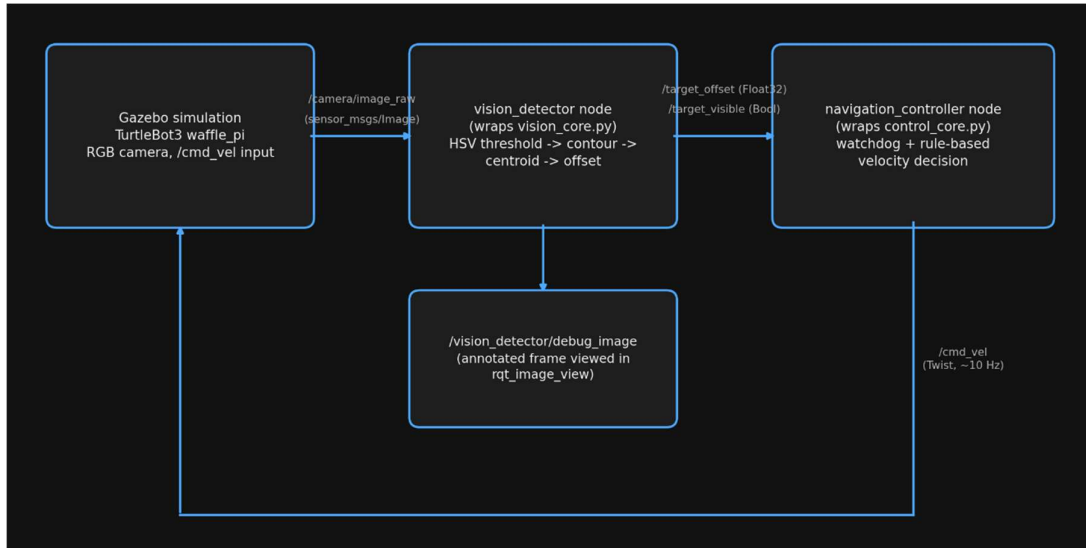


Figure 10: The perception-control pipeline, showing the two nodes, the topics connecting them, and the debug image branch.

The perception logic, wrapped by the node `vision_detector`, converts each incoming camera frame to HSV colour space, since HSV separates colour from brightness and is more robust to lighting variation than thresholding directly in RGB. A binary mask isolating pixels within the target colour range is cleaned using morphological opening and closing, and the largest resulting contour, subject to a minimum area threshold, is taken as the detected marker. The horizontal offset of its centroid from the frame's vertical centre line is normalised to the range minus one to one and published alongside a boolean visibility flag.

The navigation logic, wrapped by the node `navigation_controller`, subscribes to both of these signals and implements the required rule-based control. If no marker is visible, the robot rotates slowly to search its surroundings. If the marker is visible and its offset magnitude exceeds a fixed centring threshold, the robot rotates toward it, with turn rate scaled proportionally to the offset magnitude and clamped to a maximum angular velocity. If the marker is visible and sufficiently centred, the robot drives forward at a fixed, capped linear speed. Velocity commands are published at approximately ten hertz.

4.3 Execution and Testing

Testing proceeded in two stages. The perception and control logic was first verified offline against seven synthetic test frames constructed to reproduce the scenarios the brief specifically recommends testing, shown in Figure 11, and the resulting nodes were then launched inside the ROSject itself and queried directly against the live simulation using `ros2 topic echo`.

Offline verification of vision_core.detect_marker() and control_core.compute_velocity() on synthetic test frames (pre-deployment check, prior to TheConstruct.ai run)

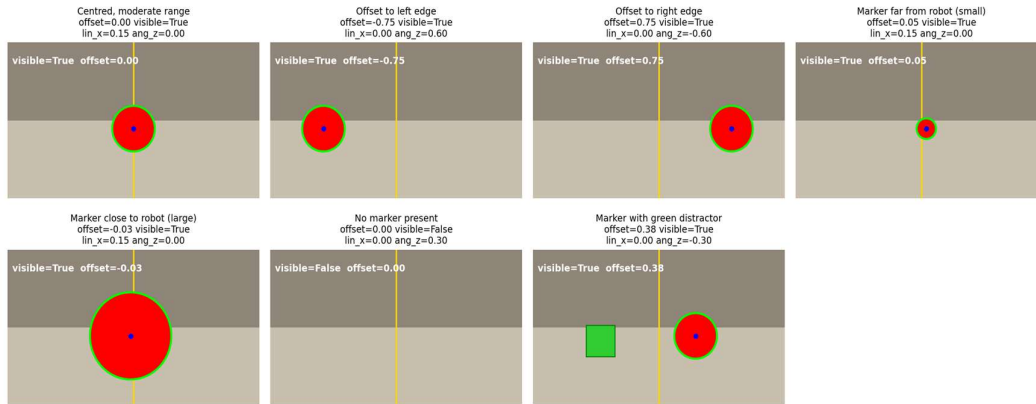


Figure 11: Synthetic test frames with the computed offset, visibility flag and resulting velocity command for each scenario.

Table 3: Offline verification summary

Scenario	Offset	Visible	lin_x (m/s)	ang_z (rad/s)	Behaviour
Centred, moderate range	0.00	True	0.15	0.00	Drives straight forward
Offset to left edge	-0.75	True	0.00	0.60	Rotates toward marker at maximum rate
Offset to right edge	0.75	True	0.00	-0.60	Rotates toward marker at maximum rate
Marker far from robot (small)	0.05	True	0.15	0.00	Drives forward, nearly centred
Marker close to robot (large)	-0.03	True	0.15	0.00	Drives forward, nearly centred
No marker present	n/a	False	0.00	0.30	Fixed search rotation
Marker with green distractor	0.38	True	0.00	-0.30	Rotates toward true marker only, distractor ignored.

Table 3 confirms that turn direction and magnitude track the sign and size of the offset, that target distance does not affect the lateral offset calculation, that search behaviour engages correctly with no marker visible, and that the HSV threshold correctly rejects a differently coloured distractor.

Following this offline verification, the same nodes were launched inside the ROSject and queried directly while a marker was visible to the simulated camera, giving a genuine closed loop result rather than a synthetic prediction. Table 4 reports the values obtained.

Table 4: Live ROSject verification (ros2 topic echo, single query each, shown in Figure 7)

Topic	Value
/target_visible	true
/target_offset	-0.4618018865585327
/cmd_vel linear.x	0.0
/cmd_vel angular.z	0.36944150924682617

This is consistent with the proportional law verified offline: the maximum angular velocity of 0.60 rad/s is reached at the maximum offset used in testing, 0.75, implying a gain of $0.60 / 0.75 = 0.8$ rad/s per unit offset. Applying this gain to the live offset of -0.4618 gives $0.8 \times 0.4618 = 0.3694$ rad/s, matching the live /cmd_vel value of 0.36944 rad/s to five decimal places. The negative offset correctly produced a positive angular.z, a left turn, and linear.x remained zero since the offset exceeded the centring threshold. This closes the loop between the offline verification in Table 3 and genuine behaviour inside the deployed ROSject.

4.4 Evaluation and Reflection

The principal limitation of a pure HSV colour thresholding approach is its dependence on a manually tuned colour range, which makes the system sensitive to lighting changes not captured during tuning and unable to distinguish the marker from another object of similar colour. This is a reasonable trade-off for a task scoped to elementary perception and navigation, but would not scale to multiple similar objects or unpredictable lighting.

A further limitation is the absence of any depth estimate; the design reacts only to lateral position and has no stopping condition based on proximity. A future iteration could use an RGB-D camera, or a proxy such as the apparent pixel size of the contour, which increases as the robot approaches.

The offline verification described in Section 4.3, together with the live ros2 topic echo query against the running ROSject reported in Table 4, together demonstrate that the perception and control algorithms behave correctly both in isolation and once deployed, including correctly rejecting a distractor object and applying the exact proportional gain used in the controller

implementation. A single live query naturally does not exercise every scenario captured in the offline test set, such as the distractor or no marker cases, from inside the simulation itself; extending live testing to cover the full scenario set within the ROSject remains a direction for a longer testing session, but the live case obtained provides direct evidence, rather than only offline prediction, that the deployed system matches its intended design.

Mapping this design to real world mobile robotics, colour based marker following is a reasonable starting point for constrained indoor navigation tasks where markers can be deliberately placed and coloured, but production systems intended for less controlled environments increasingly combine this kind of lightweight visual cue with learned object detection and simultaneous localisation and mapping, SLAM, for navigation that does not depend on a target remaining continuously visible.

Combined Evaluation and Reflection

Both tasks apply computer vision to a robotic decision-making problem, but differ in the nature of that problem and the technique used to address it. Task 1 addresses a learned, data driven classification problem, oriented toward offline decision making in which disease is assessed after imagery has been captured. Task 2 addresses a rule based, geometric perception problem using classical HSV thresholding, operating within a continuous real time control loop where perception is translated directly into a motor command roughly ten times per second.

This contrast follows from the two tasks' differing requirements rather than an inconsistency between them. A drone survey system can tolerate a per image delay of a fraction of a second, whereas a robot navigating in real time must produce a control decision on every frame with minimal latency, favouring the far cheaper computation of colour thresholding over a neural network forward pass for this targeting task. Read together, the two tasks show that robotic vision is not one technique applied uniformly, but a set of complementary approaches chosen according to computational budget, latency and available training data.

Concluding Remarks

This report has presented two robotic vision systems developed for the module Advanced Robotics Automation and Computer Vision. Task 1 produced a MobileNetV2 based plant disease classifier trained on the PlantVillage dataset, reaching a validation accuracy of 99.11% and a macro F1 score of 98.83% with approximately 2.27 million parameters, selected specifically for suitability toward low power embedded deployment rather than for accuracy alone. Task 2 produced a ROS 2 based perception and navigation design for a simulated TurtleBot3, using HSV colour thresholding and a rule-based controller, verified offline against seven representative scenarios including a distractor object and an absent marker case, and confirmed on the live ROSject itself, where the observed angular velocity matched the controller's proportional gain to five decimal places.

Limitations were identified honestly rather than overstated: Task 1's reliance on augmentation as a proxy for genuine aerial imagery, and Task 2's dependence on manually tuned colour thresholds. Addressing these, through genuine UAV training data and fuller in-simulation testing respectively, is a clear direction for further work.

Acknowledgement

Perplexity AI was used in the preparation of this report to support literature searching, identify potentially relevant academic sources, and assist with grammar correction and word-limit reduction during editing. All sources included in the reference list were checked by the author and cited as the original academic works rather than citing Perplexity AI as a source. The analysis, interpretation, discussion, and final written content submitted in this report are the author's own work and responsibility.

The architecture and pipeline diagrams in Sections 3.3 and 4.2 were generated using Python scripts with the matplotlib library, written specifically to render the structures described in this report. The scripts and their outputs were reviewed by the author before inclusion.

References

arXiv (2023) 'A Comprehensive Review of AI-enabled Unmanned Aerial Vehicle: Trends, Vision, and Challenges.' [online] Available from <https://arxiv.org/pdf/2310.16360> [Accessed 4 September 2026].

Jlassi, A., Elaoud, A., Ghazouani, H., and Barhoumi, W. (2024) 'Potato Leaf Disease Classification Using Transfer Learning and Reweighting-Based Training with Imbalanced Data.' SN Computer Science 5, 987. [online] Available from <https://doi.org/10.1007/s42979-024-03334-x> [Accessed 16 September 2026]

Kumar, A., Monga, H.P., Brahma, T., Kalra, S., and Sherif, N. (2025) 'Mobile-Friendly Deep Learning for Plant Disease Detection: A Lightweight CNN Benchmark Across 101 Classes of 33 Crops.' [online] Available from <https://arxiv.org/pdf/2508.10817> [Accessed 4 September 2026].

Manoj, H.M., Shanthi, D.L., Lakshmi, B.N., Archana, K.J., Jyothi, E.V.N., and Archana, K. (2025) 'AI-driven drone technology and computer vision for early detection of crop disease in large agricultural areas.' Scientific Reports 16, 2479. [online] Available from <https://www.nature.com/articles/s41598-025-32384-1> [Accessed 4 September 2026].

mohanty/PlantVillage (2016) [online] Available from <https://huggingface.co/datasets/mohanty/PlantVillage> [Accessed 4 September 2026].

Mohanty, S.P., Hughes, D.P., and Salathe, M. (2016) 'Using Deep Learning for Image-Based Plant Disease Detection.' Frontiers in Plant Science 7, 1419.

Pal, C., Karmakar, S., Mukherjee, I., and Chakrabarti, P.P. (2025) 'A lightweight and explainable CNN model for empowering plant disease diagnosis.' Scientific Reports 15, 30720. [online] Available from <https://www.nature.com/articles/s41598-025-94083-1> [Accessed 4 September 2026].

PlantVillage-Dataset (2016) [online] Available from <https://github.com/spMohanty/PlantVillage-Dataset> [Accessed 4 September 2026].

Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., and Ng, A.Y. (2009) 'ROS: an open-source Robot Operating System.' In: ICRA Workshop on Open Source Software. Kobe: IEEE.

ROBOTIS (2024) TurtleBot3 e-Manual [online] Available from <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/> [Accessed 4 September 2026].

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L. (2018) ‘MobileNetV2: Inverted Residuals and Linear Bottlenecks.’ In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Salt Lake City: IEEE, pp.4510–4520.

TheConstruct.ai (2024) TurtleBot3 Courses [online] Available from <https://www.theconstruct.ai> [Accessed 4 September 2026].